UNIVERSITÉ LIBRE DE BRUXELLES

Bachelier sciences informatiques

Algorithmique 2

Rapport Projet Algorithmique 2

Abdelmouhaimin Karbach, 593038

Moaad Benjana, 550644

2026

Table des matières

1 Introduction **1**

1.1 Résumé 1

2 Méthodologie **2**

2.1 Tâche 1 — Graphe non orienté (Union-Find) 2

2.2 Tâche 2 — Graphe orienté (Tarjan) 2

3 Résultats **4**

3.1 Tâche 1 — Communautés de co-publication 4

3.2 Tâche 2 — Composantes fortement connexes 5

4 Discussion **6**

4.1 Discussion 6

4.2 Conclusion 7

1 Introduction

1.1 Résumé

Ce projet vise à analyser la base de données DBLP, qui recense des publications scientifiques en informatique. L'objectif est d'identifier des communautés de chercheur·euse·s à partir de leurs collaborations.
Deux approches sont étudiées. La première repose sur un graphe non orienté de co-publication afin de regrouper les auteur·rice·s connectés. La seconde utilise un graphe orienté basé sur des relations de collaboration fortes pour extraire des communautés fortement connexes.
Le projet impose un traitement en flux des données, nécessitant des algorithmes efficaces en temps et en mémoire. Il permet ainsi de mettre en pratique les notions d'algorithmique des graphes et de structures de données vues au cours.

2 Méthodologie

2.1 Tâche 1 — Graphe non orienté (Union-Find)

Pour la tâche 1, les relations de co-publication sont modélisées à l'aide d'un graphe non orienté où chaque sommet représente un auteur et chaque arête une collaboration entre deux auteur·rice·s apparaissant dans une même publication. Afin de satisfaire la contrainte de traitement en flux, les communautés sont maintenues dynamiquement au fur et à mesure de la lecture des données.

Pour cela, une structure de données de type Union-Find (Disjoint Set Union) est utilisée. Chaque auteur est associé à un représentant, et les opérations de fusion permettent de regrouper efficacement les auteurs appartenant à une même publication. Avant chaque fusion, les noms d'auteurs sont dédupliqués afin de gérer les cas de répétition dans une même publication.

À chaque nouvelle publication, tous les co-auteur·rice·s sont unifiés dans une même composante connexe. Cette approche permet de maintenir en permanence les communautés ainsi que leur taille, grâce à une structure auxiliaire stockant la taille de chaque ensemble. Des statistiques intermédiaires sont affichées régulièrement, notamment le nombre total de communautés et la taille des plus grandes, ce qui permet de vérifier le bon fonctionnement du traitement en ligne.

En termes de complexité, les opérations de Union-Find s'exécutent en temps amorti $O(\alpha(n))$ grâce à la compression de chemin et à l'union par rang, α étant la fonction inverse d'Ackermann, quasi-constante en pratique. La complexité totale pour n auteurs et p publications est donc $O(p \cdot \alpha(n))$ en temps et $O(n)$ en mémoire.

2.2 Tâche 2 — Graphe orienté (Tarjan)

Pour la tâche 2, pour chaque publication ayant au moins deux auteurs, on incrémente un compteur pour la paire ordonnée $(A \to B)$, où A est le premier auteur et B chacun des autres. Ce comptage est effectué en ligne, publication par publication. Une fois le parsing terminé, on construit le graphe orienté filtré en ne conservant que les arêtes dont le compteur est ≥ 6. Ce seuil vise à ne retenir que les relations de collaboration fortes et répétées, afin de faire émerger des communautés significatives.

Les communautés sont ensuite détectées à l'aide de l'algorithme de Tarjan, qui identifie les composantes fortement connexes (CFC) du graphe filtré. Une CFC correspond à un groupe d'auteurs où chaque auteur peut atteindre tous les autres via des chemins orientés. Le choix de Tarjan est justifié par sa complexité linéaire $O(V+E)$, où V est le nombre de sommets et E le nombre d'arêtes du graphe filtré, ce qui le rend adapté à des graphes de grande taille.

Pour chacune des 10 plus grandes CFC, le diamètre est calculé à l'aide d'un BFS depuis chaque nœud de la communauté, en restant dans le sous-graphe induit par celle-ci. La complexité de ce calcul est $O(|C| \cdot (|V_C| + |E_C|))$ pour une communauté C, ce qui est acceptable étant donné la petite taille des CFC obtenues. En mémoire, la tâche 2 nécessite $O(V+E)$ pour le graphe filtré, et $O(V^2)$ dans le pire des cas pour les compteurs de paires $(A \to B)$ stockés dans une `HashMap` imbriquée.

3 Résultats

3.1 Tâche 1 — Communautés de co-publication

Après lecture de l'ensemble des publications DBLP, le programme identifie un total de 169 659 communautés. La distribution des tailles suit une loi de puissance typique des réseaux sociaux scientifiques, avec une grande majorité de petites communautés et une composante géante dominante. On dénombre 104 904 singletons, c'est-à-dire des auteurs n'ayant co-publié avec aucun autre auteur dans la base. La composante géante regroupe quant à elle 3 734 901 auteurs, ce qui représente la grande majorité des auteurs actifs de DBLP. Ce résultat est cohérent avec la littérature sur les réseaux de co-publication, où il est bien établi qu'une composante géante émerge rapidement dès lors que le réseau atteint une certaine densité. La Figure 3.1 présente l'histogramme complet en échelle log-log, permettant de visualiser à la fois les singletons, les communautés intermédiaires et la composante géante.

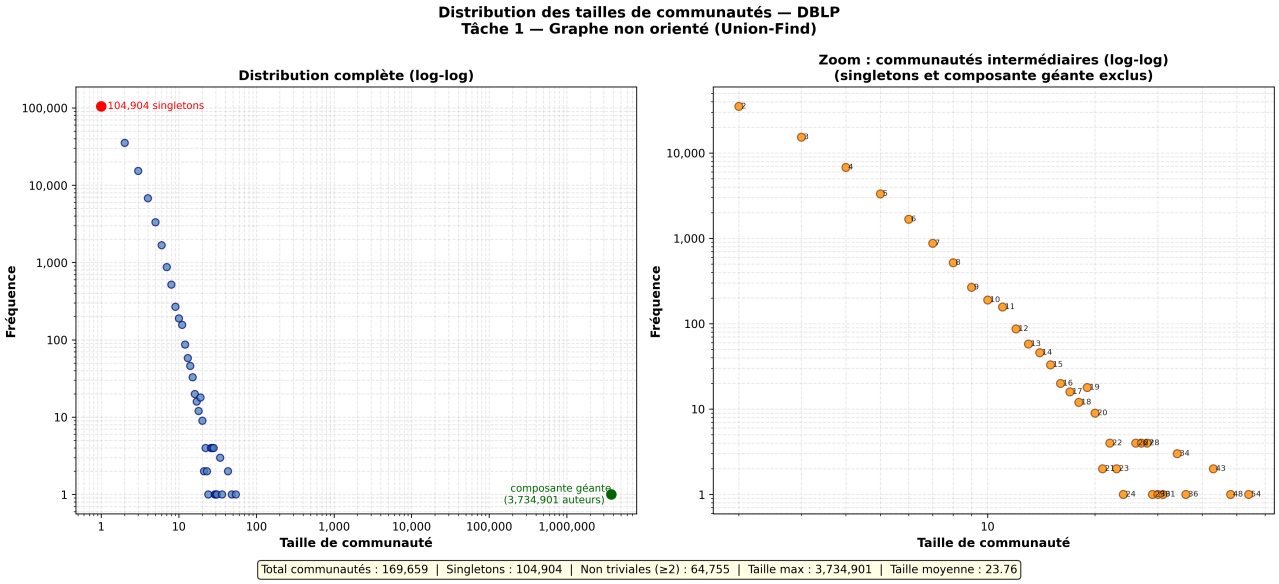

FIGURE 3.1: Distribution des tailles de communautés — Tâche 1 (Union-Find, graphe non orienté)

3.2 Tâche 2 — Composantes fortement connexes

Après construction du graphe orienté et application du seuil ≥ 6, le programme détecte 260 211 composantes fortement connexes au total. Parmi celles-ci, 10 sont non triviales avec des tailles allant de 21 à 133 auteurs. Le Tableau 3.1 présente les 10 plus grandes CFC avec leur taille et leur diamètre respectif.

Rang	Taille	Diamètre
1	133	30
2	73	35
3	38	13
4	34	7
5	33	6
6	30	17
7	27	10
8	26	12
9	23	6
10	21	7

TABLE 3.1: Les 10 plus grandes composantes fortement connexes (graphe orienté, seuil ≥ 6)

On remarque que la deuxième plus grande CFC (73 auteurs) présente un diamètre de 35, supérieur à celui de la première (133 auteurs, diamètre 30), ce qui indique une structure interne plus allongée malgré une taille moindre. De manière générale, les diamètres obtenus restent faibles au regard de la taille des communautés, ce qui confirme le phénomène de "petit monde" au sein de ces groupes de chercheurs, où tout auteur peut atteindre tout autre en peu d'étapes.

4 Discussion

4.1 Discussion

Les résultats obtenus sont globalement conformes à nos attentes. La composante géante de la tâche 1 regroupe une très grande partie des auteurs de DBLP, ce qui est cohérent avec le fait que la recherche scientifique est un domaine très interconnecté, où les collaborations forment rapidement un réseau dense. Le fait que 104 904 auteurs restent isolés s'explique principalement par des auteurs ayant publié seuls ou dans des types de publications non pris en compte par le générateur.

Pour la tâche 2, le seuil de 6 joue un rôle déterminant dans la structure des CFC obtenues. Un seuil plus bas produirait des composantes beaucoup plus grandes et moins significatives, tandis qu'un seuil plus élevé fragmenterait davantage le graphe. Le fait que les 10 plus grandes CFC restent de taille modeste (maximum 133 auteurs) montre que les relations "fortes" au sens de ce projet, c'est-à-dire un premier auteur ayant co-publié au moins 6 fois avec le même co-auteur dans le même sens, sont relativement rares dans DBLP.

Notre implémentation présente cependant quelques limites. Premièrement, l'algorithme de Tarjan utilisé pour détecter les CFC est implémenté de façon récursive, ce qui peut poser des problèmes de dépassement de pile (`StackOverflowError`) sur des graphes très denses. Une implémentation itérative serait plus robuste. Deuxièmement, le calcul du diamètre par BFS depuis chaque nœud de la communauté a une complexité de $O(|C| \cdot (|V_C| + |E_C|))$ pour une communauté C, ce qui reste acceptable pour les petites CFC obtenues ici, mais deviendrait problématique pour des communautés de taille plus importante.

Si nous devions traiter une base de données nettement plus grande comme OpenAlex (environ 480 millions de publications et 120 millions d'auteurs), notre approche actuelle ne serait pas viable. Pour la tâche 1, l'Union-Find passerait à l'échelle sans trop de difficultés étant donné sa complexité quasi-linéaire, mais la mémoire nécessaire pour stocker l'ensemble des auteurs sous forme de `String` deviendrait problématique. Une alternative serait d'encoder les auteurs par des identifiants entiers dès le parsing. Pour la tâche 2, stocker les compteurs de paires $(A \to B)$ dans une `HashMap` imbriquée consommerait une quantité de mémoire énorme. Une alternative serait d'utiliser une base de données clé-valeur sur disque, ou de traiter les données par blocs avec une approche de type MapReduce.

4.2 Conclusion

Ce projet nous a permis de mettre en pratique les notions de composantes connexes et fortement connexes sur un jeu de données réel et volumineux. Les deux tâches ont été implémentées en flux, conformément aux contraintes de l'énoncé, à l'aide de l'Union-Find pour la tâche 1 et de l'algorithme de Tarjan pour la tâche 2. Les résultats obtenus sont cohérents avec les propriétés connues des réseaux de co-publication scientifique, notamment l'émergence d'une composante géante en tâche 1 et la rareté des relations fortes en tâche 2.

Les principales limites de notre implémentation sont le risque de dépassement de pile de Tarjan récursif et la consommation mémoire élevée des `HashMap` imbriquées pour la tâche 2. Ces deux points seraient les premiers à traiter dans une version plus robuste du programme. Enfin, le passage à l'échelle vers des bases de données de la taille d'OpenAlex nécessiterait une refonte plus profonde, notamment avec un encodage entier des auteurs et un stockage sur disque des compteurs de paires.